

What Is java?

Java is a popular Internet programming language, created in 1995. It is owned by Oracle (Previously owned by Sun Micro Systems), billions of devices run Java.

It is used for:

- Mobile applications (especially Android apps)
- Desktop applications
- Web applications
- Web servers and Application servers
- Games
- Database connection

Why Java?

Java is one of the most popular and widely used programming language and platform. A platform is an environment that helps to develop and run programs written in any programming language. Java is fast, reliable and secure. From desktop to web applications, scientific supercomputers to gaming consoles, cell phones to the Internet, Java is used in every nook and corner.

History of java



- **James Arthur Gosling**, (born May 19, 1955) is a Canadian computer scientist, best known as the founder and lead designer behind the Java programming language.
- **Java** was started as a project called "Oak" by James Gosling in June 1991 at Sun Microsystems of USA.
- The **language** was initially called Oak after an **oak tree** that stood outside Gosling's office.

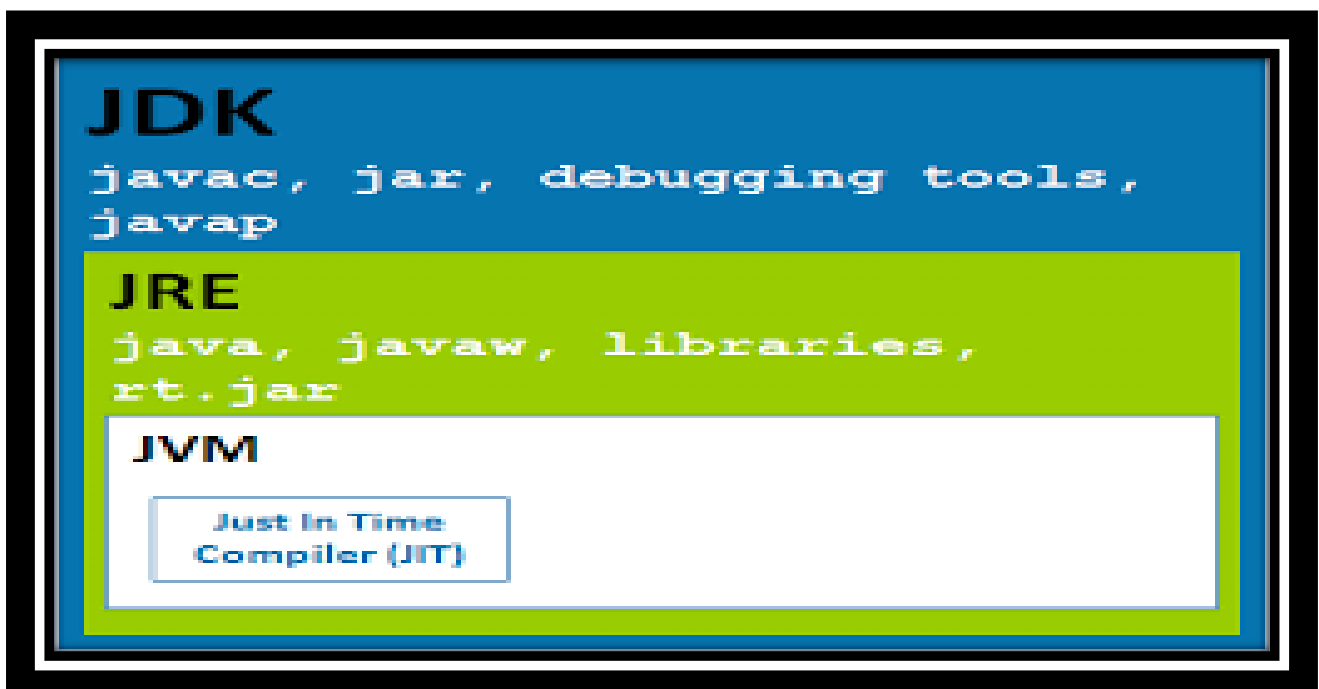
Later the project went by the **name Green** and was finally renamed **Java**, from **Java** coffee.

"**Java**" being used as slang for "coffee".
- Gosling designed **Java** with a C/C++-style syntax that system and application programmers would find familiar.
- The first public implementation was **Java 1.0** in 1995.

Major Milestones & Version History

- **JDK 1.0 (January 1996):** The initial public release, codenamed Oak.
- **JDK 1.1 (February 1997):** Added inner classes and JDBC.
- **J2SE 1.2 (December 1998):** Introduced Swing, collections framework, and JIT compiler (renamed Java 2).
- **J2SE 1.3 (May 2000):** Codenamed Kestrel, included the HotSpot JVM.
- **J2SE 1.4 (February 2002):** Added regular expressions, chained exception, and NIO.
- **J2SE 5.0 (September 2004):** Codenamed Tiger, brought generics, annotations, and enums.
- **Java SE 6 (December 2006):** Codenamed Mustang, focused on performance and scripting support.
- **Java SE 7 (July 2011):** Added switch with strings, try-with-resources, and the Fork/Join framework.
- **Java SE 8 (March 2014) [LTS]:** Major update featuring Lambda expressions and the Stream API.
- **Java SE 9 (September 2017):** Introduced the Java Platform Module System (Project Jigsaw).
- **Java SE 10 (March 2018):** Brought local-variable type inference (var).
- **Java SE 11 (September 2018) [LTS]:** Added a standard HTTP Client API.
- **Java SE 12–16 (2019–2021):** Rapid 6-month feature releases introducing pattern matching previews, records, and foreign-function interfaces.
- **Java SE 17 (September 2021) [LTS]:** Important long-term support release with enhanced performance and security.
- **Java SE 18–20 (2022–2023):** Feature updates like vector API increments and structured concurrency previews.
- **Java SE 21 (September 2023) [LTS]:** Long-term support with virtual threads and sequenced collections.
- **Java SE 22 & 23 (2024):** Refined string templates, unnamed variables, and primitive types in patterns.
- **Java SE 24 & 25 (2025):** Continued incubator updates and finalized Java SE 25 (LTS) in September 2025.
- **Java SE 26 (March 2026):** Current feature release adding modern platform enhancements

Java Terminology:



Java Development Kit (JDK):

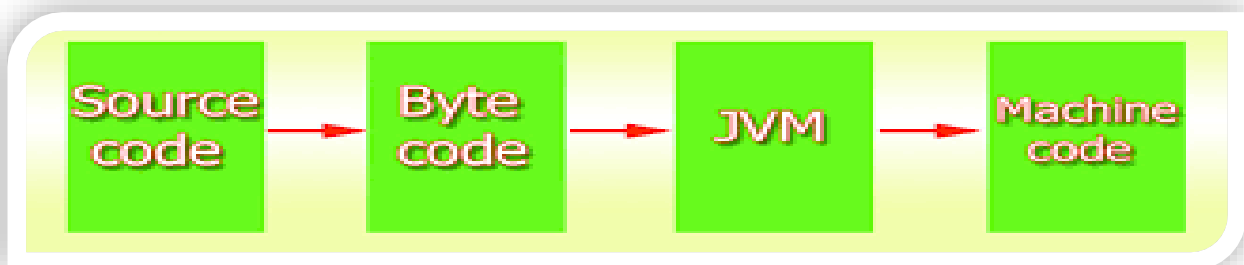
- It is a software development environment used for developing Java applications.
- JDK is a kit (or package) that includes two things:
- DevelopmentTools (provide an environment to develop your java programs)
- JRE (to execute your java program).
- For the program to execute in java, we need to download and install JDK on our computer to create, compile and run the java program.

Java Runtime Environment (JRE):

- JDK includes JRE.
- It is an installation package that provides an environment to **only run (not develop)** the java program (or application) onto your machine.
- JRE includes a browser, JVM, applet supports, and plugins.
- For running the java program, a computer needs JRE:

JVM (JavaVirtual Machine):

- JVM is a engine that provides runtime environment to drive the Java Code or applications. It converts Java bytecode into machines language.
- JVM is a part of JRE (Java Run Environment). It stands for JavaVirtual Machine
- First, Java code is compiled into bytecode. This bytecode gets interpreted on different machines
- Between host system and Java source, Bytecode is an intermediary language.
- JVM is responsible for allocating memory space.

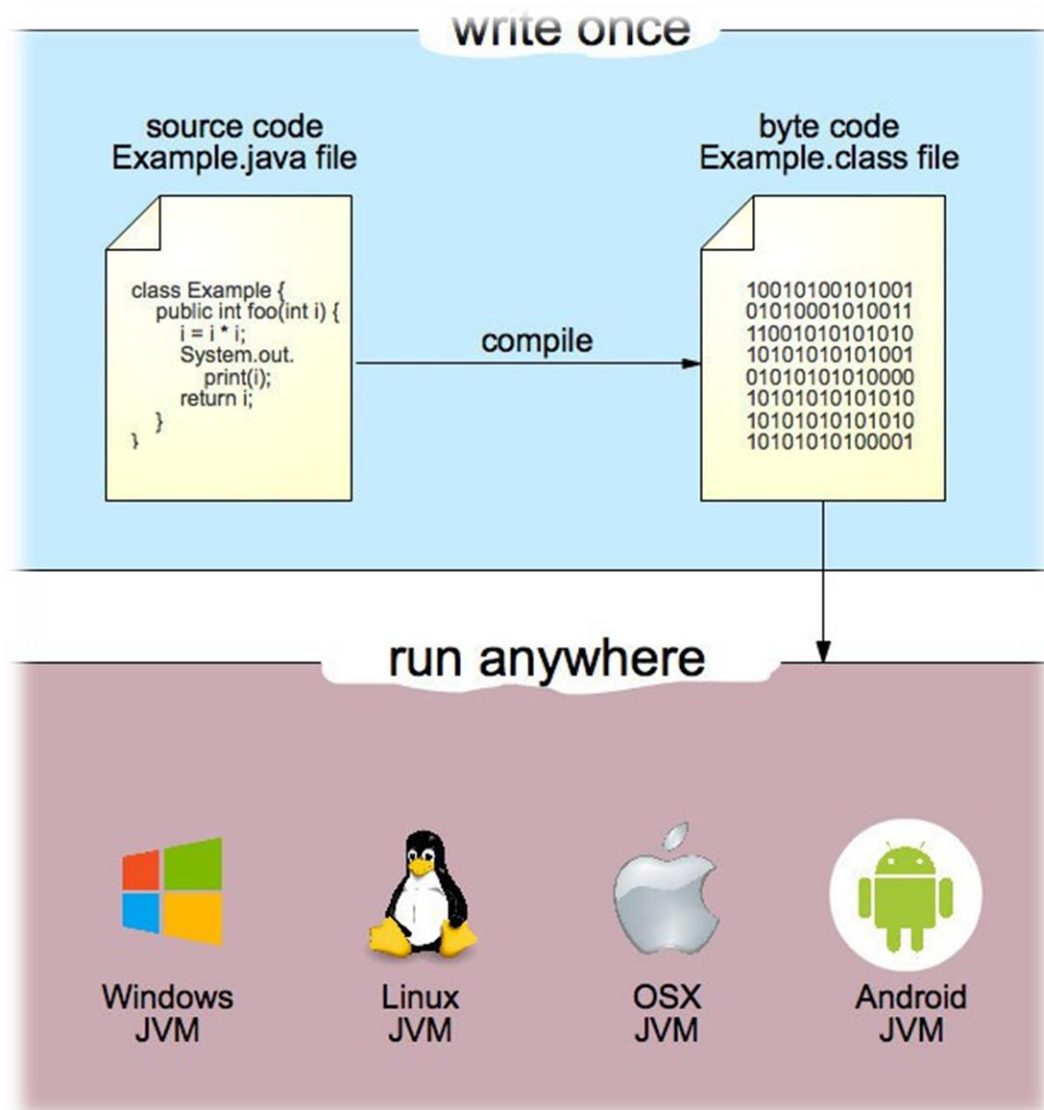


The Byte Code:

- Bytecode in Java is an intermediate machine-independent code.
- In general, bytecode is a code that lies between low-level and high-level language.
- The bytecode is not processed by the processor. It is processed by the **Java Virtual Machine (JVM)**. It is a set of instructions for JavaVirtual Machine.
- The job of the JVM is to call all the required resources to compile the Java program and make the bytecode independent.
- It is the biggest reason why java is known as a **platform-independent language**.
- The intermediate code can run on any of the platforms such as Windows, macOS, and Linux.

WORA (Write Once Run Anywhere):

“Write Once, Run Anywhere” (WORA) is a 1995 slogan created by Sun Microsystems to describe the cross-platform capability of the **Java** programming language. It means a developer can write and compile Java code on one operating system (e.g., Windows) and run it on any other hardware or operating system (e.g., macOS, Linux) without making any modifications.



Java is both Compiled & Interpreted Language

- A compiler is a program which converts a program from one level of language to another. Example conversion of C++ program into machine code.
- The java compiler converts high-level java code into bytecode (which is also a type of machine code).
- An interpreter is a program which converts a program at one level to another programming language at the same level. Example conversion of Java program into C++
- In Java, the Just InTime Code generator converts the bytecode into the native machine code which are at the same programming levels.
- Hence, Java is both compiled as well as interpreted language.
- JIT or Just-in-time compiler is the part of the JavaVirtual Machine (JVM). It is used to speed up the execution time
- In comparison to other compiler machines, Java may be slow in execution.

Features of Java (or) Java Buzzwords :

1. **Simple:**

- Java is very easy to learn, and its syntax is simple (based on C++), clean, and easy to understand.
- Java has removed many complicated and rarely used features, for example, explicit pointers, operator overloading, etc.
- There is no need to remove unreferenced objects because there is an Automatic Garbage Collection in Java.

2. **Object-oriented:**

- Java is an object-oriented programming language. Everything in Java is an object.
- Object-oriented means we organize our software as a combination of different types of objects that incorporate both data (Data Members) and behavior (Member Functions).

3. **Portable:**

- Java is portable because it facilitates you to carry the Java bytecode to any platform. It does not require any implementation.

4. **Platform independent:**

- A platform is the hardware or software environment in which a program runs.
- Java is platform independent because it is different from other languages like C, C++, etc. which are compiled into platform specific machines while Java is a **Write Once, Run Anywhere** language.
- Java code can be executed on multiple platforms, for example, Windows, Linux, Sun Solaris, Mac/OS, etc.

5. **Robust:** Java is robust (Strong) because.

- It uses strong memory management.
- There is a **lack of pointers** that avoids security problems.
- Java provides **automatic garbage collection** which runs on the Java Virtual Machine to get rid of objects which are not being used by a Java application anymore.
- There are **exception handling** and the **type checking** mechanism in Java. All these points make Java robust.

Object Oriented Programming Through Java (Unit-I)

6. Secured:

- Java is best known for its security. With Java, we can develop virus-free systems.
- Java is secured because Java Programs run inside a **virtual machine sandbox**.
- **ClassLoader** in Java is a part of the Java Runtime Environment (JRE) which is used to load Java classes into the Java Virtual Machine dynamically.
- It adds security by separating the package for the classes of the local file system from those that are imported from network sources.

7. Architecture neutral:

- Java is architecture neutral because there are no implementation dependent features, for example, the size of primitive types is fixed.
- In C programming, "int" data type occupies 2 bytes of memory for 32-bit architecture and 4 bytes of memory for 64-bit architecture. However, it occupies 4 bytes of memory for both 32 and 64-bit architectures in Java.

8. Interpreted:

- Java can be considered both a **compiled** and an **interpreted** language because its source code is first compiled into a binary bytecode.
- This byte-code runs on the Java Virtual Machine (JVM), which is usually a software-based interpreter.
- The **Java interpreter** converts the Java bytecode (.class file) into the code understood by the operating system.
- Java interpreter is a computer program (system software) that implements the JVM. It is responsible for reading and executing the program.
- It is designed in such a way that it can read the source program and translate the source code instruction by instruction. It converts the high-level program into assembly language (machine language).

9. High Performance:

- Java is faster than other traditional interpreted programming languages because Java bytecode is "close" to native code.
- It is still a little bit slower than a compiled language (e.g., C++). Java is an interpreted language that is why it is slower than compiled languages, e.g., C, C++, etc.

10. Multithreaded:

- A thread is like a separate program, executing concurrently.
- Java programs deal with many tasks at once by defining multiple threads.
- The main advantage of multi-threading is that it does not occupy memory for each thread. It shares a common memory area.
- Threads are important for multi-media, Web applications, etc.

11. Distributed:

- Java is distributed because it facilitates users to create distributed applications in Java.
- This feature of Java makes us able to access files by calling the methods from any machine on the internet.

12. Dynamic:

- Java is a dynamic language. It supports the dynamic loading of classes.
- It means classes are loaded on demand. It also supports functions from its native languages, i.e., C and C++.
- Java supports dynamic compilation and automatic memory management (garbage collection).

Object-Oriented Programming (OOP): Basic Concepts and Principles Key

Concepts:

- **Classes and Objects**
- **Encapsulation**
- **Inheritance**
- **Polymorphism**
- **Abstraction**

1. Classes and Objects

A **class** is a blueprint for creating objects. It defines a datatype by bundling data (attributes) and methods (functions) that work on the data into one single unit.

An **object** is an instance of a class. When a class is defined, no memory is allocated until an object of that class is created.

Example 1:

```
class Car {  
    String color;  
  
    int speed;  
  
    void display() {  
        System.out.println("Color: " + color);  
        System.out.println("Speed: " + speed);  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
  
        Car myCar = new Car();  
        myCar.color = "Red";  
        myCar.speed = 120;  
        myCar.display();  
    }  
}
```

Expected Output:

Color: Red

Speed: 120

2. Encapsulation

Encapsulation is the wrapping of data (variables) and code (methods) together into a single unit, usually a class.

Example:

```
class Account {  
    private double balance;  
    public void deposit(double amount) {  
  
        balance += amount;  
    }  
    public double getBalance() {  
        return balance;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
  
        Account acc = new Account();  
        acc.deposit(1000);  
        System.out.println("Balance: " + acc.getBalance());  
    }  
}
```

Expected Output:

Balance: 1000.0

3. Inheritance

Inheritance allows a new class to inherit the properties and methods of an existing class. The new class is called a **subclass** (or derived class), and the existing class is called a **superclass** (or base class).

Example 1:

```
class Animal {  
  
    void eat() {  
        System.out.println("Eating...");  
    }  
}
```

Object Oriented Programming Through Java (Unit-I)

```
}  
  
class Dog extends Animal {  
  
    void bark() {  
        System.out.println("Barking...");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
  
        Dog dog = new Dog();  
        dog.eat();  
        dog.bark();  
    }  
}
```

Expected Output:

Eating...

Barking...

4. Polymorphism

Polymorphism is a concept where more than one function will be defined with same name. There are two types of polymorphism in Java: **compile-time** (method overloading) and **runtime** (method overriding).

Method Overloading: Defining multiple methods in the same class with the same name but different no of parameters or different type of parameters is known to be method overloading.

Key Points:

- Methods must differ in the number of parameters, type of parameters, or both.
- Overloading is resolved at compile time.

Example :

```
class MathOperations {  
  
    int add(int a, int b) {  
        return a + b;  
    }  
    int add(int a, int b, int c) {  
        return a + b + c;  
    }  
}
```

Object Oriented Programming Through Java (Unit-I)

```
}

public class Main {

    public static void main(String[] args) {

        MathOperations math = new MathOperations();

        System.out.println("Sum: " + math.add(5, 10));

        System.out.println("Sum: " + math.add(5, 10, 15));

    }

}
```

Expected Output:

Sum: 15

Sum: 30

Method Overriding

When a Child class provides implementation for a method which is already defined in its superclass is known to be method overriding. The method must have the same name, return type, and parameters. Overriding is resolved at runtime.

Example :

```
class Animal {

    void sound() {

        System.out.println("Animal makes a sound");

    }

}

class Dog extends Animal {

    void sound() {

        System.out.println("Dog barks");

    }

}

public class Main {

    public static void main(String[] args) {

        Animal myDog = new Dog();

        myDog.sound();

    }

}
```

```
}  
}
```

Expected Output:

Dog barks

5. Abstraction

Abstraction is the concept of hiding the complex implementation details and showing only the essential features of the object. In Java, abstraction is achieved using **abstract classes** and **interfaces**.

Example 1: (Abstract Class)

```
abstract class Animal {  
  
    abstract void sound();  
  
    void eat() {  
        System.out.println("Eating...");  
    }  
}  
  
class Dog extends Animal {  
  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
  
        Dog dog = new Dog();  
        dog.sound();  
        dog.eat();  
    }  
}
```

Expected Output:

Dog barks

Eating...

Example 2: (Interface)

```
interface Drawable {  
    void draw();  
}  
  
class Circle implements Drawable {  
  
    public void draw() {  
        System.out.println("Drawing Circle");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
  
        Drawable circle = new Circle();  
        circle.draw();  
    }  
}
```

Expected Output

Drawing Circle

Java Program Structure :

A Java program typically consists of several parts that are organized into a specific structure. Below is a breakdown of the basic components of a Java program.

1. Package Declaration

- A package is an organization of set of related classes and interfaces.
- The package statement, if present, should be the first line of the source file.

Example : package com.example.myapp;

2. Import Statements

- Import statements are used to make other Java classes or entire packages available to your class so that you can use them.

Example:

```
import java.util.Scanner;
```

Object Oriented Programming Through Java (Unit-I)

```
import java.util.List;
```

3. Class Declaration

- The class declaration is the main part of the Java program. It is the blueprint for objects.
- A Java program must have at least one class, and this class must contain the main method if it's a standalone application.

Example:

```
public class MyClass {  
    // Class body  
}
```

4. Main Method

- The main method is the entry point of execution.
- It's always public, static, returns void, and takes a String[] argument.

Syntax:

```
public static void main(String[] args) {  
    // Code to be executed  
}
```

5. Instance Variables and Methods

- **Instance Variables:** These are variables that belong to an instance of a class (i.e., to an object).
- **Methods:** Functions will be defined inside a class to perform specific tasks.

Example:

```
public class MyClass {  
    // Instance variable  
  
    int number;  
  
    // Method  
    void displayNumber() {  
        System.out.println("Number: " + number);  
    }  
}
```

6. Constructors

Object Oriented Programming Through Java (Unit-I)

- A constructor is a special method that is called when an object is instantiated(created).
- It has the same name as the class and does not have a return type.

Example:

```
public class MyClass {  
  
    int number;  
  
    // Constructor  
    MyClass(int num) {  
        number = num;  
    }  
}
```

*** Note : We can define constructor with any access specifier public, protected, default and private but return type can not be specified for a constructor.**

7. Comments

- Comments are non-executable parts of the code that are used to provide explanations or annotations.

Types of Comments:

- **Single-line comments:** // This is a single-line comment
- **Multi-line comments:** /* This is a multi-line comment */
- **Documentation comments:**
/** * This is a documentation comment.
* It is used to generate documentation. */

Example: Complete Java Program

```
package com.example.myapp; // Package declaration import  
  
java.util.Scanner; // Import statement  
  
public class MyClass {    // Class declaration  
    int number;          // Instance variable  
  
    // Constructor  
    public MyClass(int num) {  
        number = num;  
    }  
}
```

Object Oriented Programming Through Java (Unit-I)

```
// Method
void displayNumber() {
    System.out.println("Number: " + number);
}

// Main method
public static void main(String[] args) {
    MyClass obj = new MyClass(10); // Create an object
    obj.displayNumber();           // Call a method
}
}
```

Note :

- **Package declaration** (optional) if written should be first statement.
- **Import statements** (optional).
- The **class declaration** defines the structure and behaviour of objects.
- **Main method** is the entry point for the program.
- **Instance variables** and **methods** define the properties and behaviour of objects.
- **Constructors** initialize objects.
- **Comments** are used for explanations.

Elements or Tokens in Java Programs

In Java, tokens are the smallest units or building blocks of a program. They are the basic components that the Java compiler recognizes. They consist of following

1. Keywords

- **Definition:** Reserved words in Java will have a specific meaning and cannot be used as identifiers like variable names, function names.
- **Examples:** int, class, void, if, else, while, return, public, static.

Example:

```
public class MyClass {
    int number;
}
```

Here, public, class, and int are keywords.

2. Identifiers

- **Definition:** Names given to various elements of a program such as variables, methods, classes, and objects.

Object Oriented Programming Through Java (Unit-I)

- **Rules:**

- Must begin with a letter (A-Z or a-z), a dollar sign (\$), or an underscore (_).
- Cannot start with a digit.
- Cannot be a keyword.
- Case-sensitive (MyClass and myclass are different).

Example:

```
public class MyClass {  
  
    int number;  
  
    void displayNumber() {  
        System.out.println(number);  
    }  
}
```

Here, MyClass, number, and displayNumber are identifiers.

3. Literals

- **Definition:** values that appear directly in the code.
- **Types:**
 - **Integer literals:** Numbers without a decimal point (e.g., 10, 0, -5).
 - **Floating-point literals:** Numbers with a decimal point (e.g., 3.14, -0.01).
 - **Character literals:** Single characters enclosed in single quotes (e.g., 'A', '3').
 - **String literals:** Sequence of characters enclosed in double quotes (e.g., "Hello", "123").
 - **Boolean literals:** true or false.
 - **Null literal:** null.

Example:

```
int age = 25; char  
initial = 'A';  
String name = "John";  
boolean isActive = true;
```

Here, 25, 'A', "John", and true are literals.

4. Operators

- **Definition:** Symbols that perform operations on variables and values.
- **Categories:**
 - **Arithmetic operators:** +, -, *, /, %.

Object Oriented Programming Through Java (Unit-I)

- **Relational operators:** ==, !=, >, <, >=, <=.
- **Logical operators:** &&, ||, !.
- **Assignment operators:** =, +=, -=, *=, /=.
- **Unary operators:** ++, --.
- **Ternary operator:** ? :.

Example:

```
int a = 10, b = 5;
```

```
int sum = a + b; // Arithmetic operator
```

```
boolean result = a > b; // Relational operator
```

Here, +, =, and > are operators.

5. Separators (Delimiters)

- **Definition:** Characters that separate tokens in the code.
- **Examples:**
 - **Parentheses ()** : Used in method declarations and to group expressions.
 - **Braces {}** : Define the beginning and end of blocks of code.
 - **Brackets []** : Used in arrays to define the size and access elements.
 - **Semicolon ;** : Terminates statements.
 - **Comma ,** : Separates variables in declarations and parameters in method calls.
 - **Dot .** : Access members of a class or object.

Example:

```
int[] numbers = {1, 2, 3}; // Braces, brackets, semicolon
```

```
System.out.println(numbers[0]); // Parentheses, dot, brackets, semicolon
```

Here, {}, [], (), ., and ; are separators.

6. Comments

- **Definition:** Non-executable parts of the code that are used to explain the code or leave notes for other developers.
 - **Types:**
 - **Single-line comment:** Starts with //.
 - **Multi-line comment:** Enclosed within /* and */.
 - **Documentation comment:** Starts with /** and is used to generate documentation. */

Example:

Object Oriented Programming Through Java (Unit-I)

```
// This is a single-line comment
```

```
/*  
This is a  
multi-line comment  
*/
```

```
/**  
* This is a documentation comment  
*/
```

Note:

- **Keywords:** Reserved words with special meaning.
- **Identifiers:** Names for variables, methods, classes, etc.
- **Literals:** values in the code (e.g., numbers, characters, strings).
- **Operators:** Symbols that perform operations on variables and values.
- **Separators:** Characters that organize code (e.g., {}, [], ;, .).
- **Comments:** Notes and explanations in the code, not executed by the compiler.

Java Statements:

Java statements are instructions that the compiler executes. They include:

1. Expression Statements: Perform operations like assignments or method calls.
2. Declaration Statements: Declare variables or constants.
3. Control Flow Statements: Manage the flow of execution (if, for, while).
4. Block Statements: Group multiple statements with {}.

Command Line Arguments in Java :

- Command line arguments are values passed to a Java program at the time of execution from the command line. These arguments are passed as a sequence of strings and are stored in the **String[] args** parameter of the **main** method.
- Command line arguments are passed when you run the Java program using the java command followed by the class name and arguments.

Example:

Object Oriented Programming Through Java (Unit-I)

```
java MyClass arg1 arg2 arg3
```

In this example, `arg1`, `arg2`, and `arg3` are command line arguments.

Accessing Command Line Arguments

- The arguments are accessed within the `main` method using the `args` array.
- The `args` array is zero-indexed, meaning `args[0]` is the first argument, `args[1]` is the second, and so on.
- The length of the `args` array can be determined using `args.length`.

Example:

```
public class MyClass {  
    public static void main(String[] args)  
    {  
        System.out.println("Number of arguments: " + args.length);  
        for (int i = 0; i < args.length; i++)  
        {  
            System.out.println("Argument " + i + ": " + args[i]);  
        }  
    }  
}
```

If this program is run with the command:

```
java MyClass Hello World 2024
```

The output will be:

```
Number of arguments: 3  
Argument 0: Hello  
Argument 1: World  
Argument 2: 2024
```

Uses of Command Line Arguments

- **Passing configuration options:** Command line arguments are often used to pass configuration parameters to a program, such as file names, URLs, or settings.

Example: Sum of Numbers Passed as Arguments:

```
public class SumArguments {  
    public static void main(String[] args) {  
        int sum = 0;  
  
        for (String arg : args) {  
  
            //Integer.parseInt() -Convert number given in String format to integer  
            //i.e., string "53" will be converted to number 53  
  
            sum += Integer.parseInt(arg);  
        }  
  
        System.out.println("Sum of arguments: " + sum);  
    }  
}
```

If run with:

```
java SumArguments 10 20 30
```

The output will be:

```
Sum of arguments: 60
```

User Input to Programs:

The user input can be read using the classes like Scanner, BufferedReader, Console, StreamTokenizer, etc.

The **Scanner** class in Java is a part of the java.util package. It is used to read input of various types (like strings, integers, and doubles) from different input sources such as the keyboard (standard input), files.

Key Features of the **Scanner** Class:

1. **Input Sources:**

- The Scanner class can read input from various sources:
 - **Standard Input (Keyboard):** Typically System.in.
 - **Files:** Reading from files using File objects.

2. **Methods for Different Data Types:**

- The Scanner class provides methods to read different types of data:
Below are some of the methods of Scanner class
 - **nextLine():** Reads a line of text (string).
 - **nextInt():** Reads an integer.
 - **nextDouble():** Reads a double (floating-point number).
 - **nextBoolean():** Reads a boolean value (true or false).
 - **next():** Reads a word (string up to the first whitespace).

Example of Using the Scanner Class:

```
import java.util.Scanner;
public class ScannerExample
{
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int number = scanner.nextInt();
        System.out.print("Enter a word: "); String
        word = scanner.next();
        System.out.println("You entered: " + number + " and " + word);
    }
}
```

BufferedReader class – The **BufferedReader** class in Java is used to read text from an input stream (like a file or keyboard) efficiently. It is part of the java.io package and provides buffering, which means it reads

Object Oriented Programming Through Java (Unit-I)

larger chunks of data at once and stores them in a buffer, reducing the number of I/O operations. This makes it faster, especially when reading large files or data.

Example of Using BufferedReader:

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.IOException;

class BufferedReaderExample {
    public static void main(String[] args) {
        BufferedReader reader = new BufferedReader(new InputStreamReader(System.in));
        try {
            System.out.print("Enter your name: ");
            String name = reader.readLine(); // Reads a line of text
            System.out.println("Hello, " + name + "!");
        } catch (Exception e) {
            System.out.println("Exception caught: "+e);
        }
    }
}
```

Console Class

The **Console** class in Java, part of the java.io package, is used to read input from and write output to the console. It is particularly useful in command-line applications and is designed to handle both input and output with a simple and secure interface.

Key Features of the Console Class:

- 1. Reading Input:**
 - The Console class provides methods like `readLine()` for reading a line of text and `readPassword()` for securely reading passwords (which masks the input).
- 2. Writing Output:**
 - You can use methods like `printf()` and `format()` to write formatted text to the console, similar to `System.out.printf()`.
- 3. Security:**
 - The `readPassword()` method reads input without showing it on the screen, which is useful for password input.
- 4. Limited Availability:**
 - The Console class is not available in all environments. It may return null in IDEs or environments that don't have a console, so it's often used in applications run directly from a command line.

Example of Using the Console Class:

```
import java.io.Console;

class ConsoleExample {
    public static void main(String[] args) {
        Console console = System.console();
        if (console != null) {
            String username = console.readLine("Enter your username: ");
            char[]
```

Object Oriented Programming Through Java (Unit-I)

```
password = console.readPassword("Enter your password: ");

System.out.println("Username: " + username);
System.out.println("password: " + password);
// Note: Don't print passwords in real applications
} else {
    System.out.println("No console available.");
}
}
```

Comparison of Scanner, BufferedReader, Console, and StreamTokenizer:

Feature	Scanner	BufferedReader	Console
Primary Use	Reading and parsing primitive data types and strings from input sources (keyboard, files).	Efficiently reading text line by line from any Reader (like files or console).	Interacting with the console for secure input and output.
Input Source	InputStream (e.g., System.in), files, strings	Reader objects (e.g., FileReader, InputStreamReader)	Console (typically System.console())
Feature	Scanner	BufferedReader	Console
Buffering	No internal buffering; reads input directly	Uses an internal buffer for efficient reading	No internal buffering; direct interaction with console

Escape sequences

Escape sequences in Java are special characters used within strings to represent new lines, tabs, or quotation marks that can't be typed directly.

Here are some common escape sequences:

- `\n`: Inserts a new line.
- `\t`: Adds a tab space.
- `\\`: Inserts a backslash.
- `\"`: Inserts a double quote.
- `\'`: Inserts a single quote.

Escape sequences help format text within strings. Examples for each escape sequence:

1. New Line (`\n`):

```
System.out.println("Hello\nWorld");
```

Output:

```
Hello
World
```

2. Tab (`\t`):

```
System.out.println("Hello\tWorld");
```

Output:

```
Hello  World
```

3. **Backslash (\):**

```
System.out.println("This is a backslash: \\");
```

Output:

This is a backslash: \

4. **Double Quote (\"):**

```
System.out.println("He said, \"Hello World!\"");
```

Output:

He said, "Hello World!"

5. **Single Quote (\'):**

```
System.out.println('');
```

Output:

'

Programming Style in Java

Programming style refers to the way code is written and organized, which affects readability, maintainability, and overall quality. Good programming style makes code easier to understand and work with. key aspects of good programming style in Java:

1. Consistent Naming Conventions

- **Classes:** Use CamelCase with the first letter capitalized.
Example: MyClass, StudentRecord.
- **Methods and Variables:** Use camelCase with the first letter lowercase.
Example: calculateTotal, studentName.
- **Constants:** Use uppercase with words separated by underscores.
Example: MAX_SIZE, PI.

2. Indentation and Spacing

- **Indentation:** Use consistent indentation (commonly 4 spaces) for code blocks, loops, and conditions.
- **Spacing:** Add spaces around operators (=, +, -, etc.) and after commas for better readability.

Example:

```
public class Example {  
    public static void main(String[] args) {  
        int total = 0;  
        for (int i = 0; i < 10; i++) { total += i;  
        }  
        System.out.println("Total: " + total);  
    }  
}
```

3. Comments

- **Inline Comments:** Use // for short comments explaining specific lines of code.

- **MultiLine-Block Comments:** Use `/* ... */` for longer explanations or to comment out blocks of code.
- **Documentation Comments:** Use `/** ... */` to describe the purpose of classes, methods, and fields, useful for generating documentation.

Example:

```
/**  
  
 * This class calculates the sum of numbers.  
  
 */  
  
public class Calculator {  
    // This method returns the sum of two numbers  
    public int add(int a, int b) {  
        return a + b;  
    }  
}
```

4. Code Structure

- **Methods:** Break down the program into small, reusable methods, each performing a single task.
- **Class Design:** Follow Object-Oriented principles like encapsulation, inheritance, and polymorphism.

Example:

```
public class Person {  
    private String name;  
    private int age;  
    // Constructor  
    public Person(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
    // Method to display person's info  
    public void displayInfo() {  
        System.out.println("Name: " + name + ", Age: " + age);  
    }  
}
```

5. Error Handling

- Use try-catch blocks to handle exceptions.
- Always ensure that resources are properly closed or released, preferably using try-with-resources.

Example:

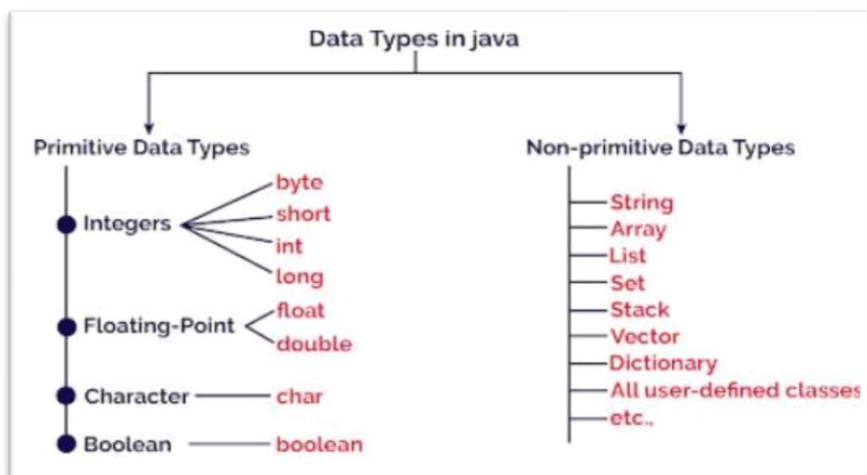
```
try {  
    int result = 10 / 0;  
} catch (ArithmeticException e) {  
    System.out.println("Cannot divide by zero!");  
}
```

Note: Good programming style in Java involves using consistent naming conventions, proper indentation and spacing, clear comments, structured code design, and effective error handling. Following these practices ensures that code is readable, maintainable, and easier to understand for others.

Data Types, Variables, and Operators:

Data Types :

Data types represent the different values to be stored in the variable. In java, there are two types of data types: Primitive data types Non-primitive data type



Name	Default Value	Size	Type
byte	0	1 byte	Integral Value
short	0	2 byte	Integral Value
int	0	4 byte	Integral Value
long	0	8 byte	Integral Value
float	0.0f	4 byte	Floating Point
double	0.0d	8 byte	Floating Point
char	'\u0000' (means 0 in ASCII)	2 byte	Character
boolean	false	1 bit	Boolean

Declaration of Variables

declaring variables is the process of specifying the variable's type and name. This tells the Java

Object Oriented Programming Through Java (Unit-I)

compiler what kind of data the variable will store.

Syntax : type variableName;

Example : int age;

 float height;

 double distance;

Type casting and Type conversion

Type Conversion (or) Automatic Type Conversion -

Also known as widening conversion, takes place when two data types can be automatically converted without any loss of data. This happens when the two data types are compatible and we assign value of a smaller data type to a larger data type.

For Example,

Example

int a;

short b = 50;

a = b; // conversion from b to a, smaller type to larger

Type Casting or Narrowing or Explicit Conversion -

If we want to assign a value stored in a larger data type variable to a smaller data type variable we perform explicit type casting or narrowing.

Example

int a = 10;

byte b;

b = (byte) a;

//we are explicitly mentioning to convert variable **a** of type **int** to **byte** and then assign to **b**

Scope of Variable Identifier :

Variable is a reference for memory location.

Types of Variables: There are three types of variables in java

- local variable
- instance variable
- static variable

1) Local Variable

A variable which is declared inside the method is called to local variable. The scope and lifetime are limited to the method itself. the arguments of a function will also be treated as local variables to that method

```
void m(int a, int b)
{
    int sum = a+b // here a,b and sum all three are local variables to function/method – m
}
```

2) Instance Variable

A variable which is declared inside the class but outside the method, is called as instance variable these should not be declared as static, because every instance of the class (object) contains its own copy of these instance variables.

The lifetime of these variables is the same as the lifetime of the object to which it belongs to. Object once created do not exist for ever. They will be destroyed by the garbage collector of Java when there are no more references to that object.

3) Static variable

A variable that is declared as static is called to be static variable. It cannot be local variable. It is a variable which belongs to the class and not to any object(instance). These variables will be initialized first, before the initialization of any instance variables.

A single copy of static variables will be shared by all instances(Objects) of the class A

static variable can be accessed directly by the class name.

Example to understand the types of variables in java

```
class A
{
    int data=50; //instance variable static
    int m=100; //static variable void
    method()
    {
        int n=90; //local variable
    }
} //end of class A
```

Literal Constants

- A literal constant refers to a constant value in the code that is fixed. In Java, it's often associated with final variables, which are constants that cannot be modified once assigned.
- Examples:

```
public class Example {
    public static final int PI =3.14; // the literal value 3.14 is called to be literal constant
}
```

Symbolic Constants :

A **symbolic constant** is a constant that is given a name in a program. Instead of using the literal value directly in the code, a name (or symbol) is defined to represent that value. This makes the code easier to read, maintain, and update because the symbolic name is used throughout the code instead of the actual literal value.

Example

```
public class Example {  
    public static final int PI = 3.14; // The variable PI is called by the symbolic constant  
                                     // The value 3.14 is called to be the literal constant  
}
```

Example

```
public class ConstantsExample {  
    public static void main(String[] args) {  
  
        // Literal example (literal value can be changed)  
        int number = 100;           // 100 is a literal  
        number = 200;               // literal 100 replaced with literal 200  
        // Literal Constant example (literal constant value cannot be changed)  
        final double LITERAL_PI = 3.14159; // 3.14159 is a literal constant  
        final int MAX_AGE = 65;           // 65 is a literal constant  
  
        // Symbolic Constant example  
        final int MAX_SIZE = 100;         // MAX_SIZE is a symbolic constant  
        final double PI = 3.14159;        // PI is a symbolic constant  
    }  
}
```

Formatted Output with printf() Method :

In Java, the `printf()` method is used to produce formatted output. It's similar to `System.out.print()` and `System.out.println()`, but with the added ability to format strings, numbers, and other types in specific ways.

- `%d` is for integers.
- `%f` is for floating-point numbers.
- `%s` is for strings.
- `%n` adds a new line.

```
public class PrintfExample {  
    public static void main(String[] args) {  
        int  
        age = 25;  
        double pi = 3.14159; String  
        name = "Alice"; int id = 123;  
        String product = "Laptop"; double  
        price = 799.99;  
        // Printing an integer
```

Object Oriented Programming Through Java (Unit-I)

```
System.out.printf("Age: %d years\n", age);
// Printing a floating-point number with 2 decimal places
System.out.printf("Value of PI: %.2f\n", pi);
// Printing a string
System.out.printf("Hello, %s!\n", name);

// Printing multiple values
System.out.printf("Product ID: %d\nProduct Name: %s\nPrice: $%.2f\n", id, product, price);
}
}
```

OUTPUT :

Age: 25 years Value of
PI: 3.14 Hello, Alice!
Product ID: 123
Product Name: Laptop Price:
\$799.99

Note:

A method will not be overloaded by return type of that method, for example if two methods have same name, same parameters and have different return type, then this is not a valid method overloading. It will be a compilation error “method has been already defined”.

Static keyword:

When a member is declared static, it can be accessed before any objects of its class are created. We can apply static keyword with variables, methods, blocks and nested class.

why static variables?

write a program to define student class with properties name, rollnum, teacher, branch. create 3 student objects with different properties initialized through a constructor. print all student details.

```
public class StaticExample
{
    String name;
    int roll_num;
    String teacher;
    String branch;

    StaticExample(String name, int roll_num, String teacher, String branch)
    {
        this.name = name;
        this.roll_num = roll_num;
        this.teacher = teacher;
        this.branch = branch;
    }

    void displayDetails()
```

Object Oriented Programming Through Java (Unit-I)

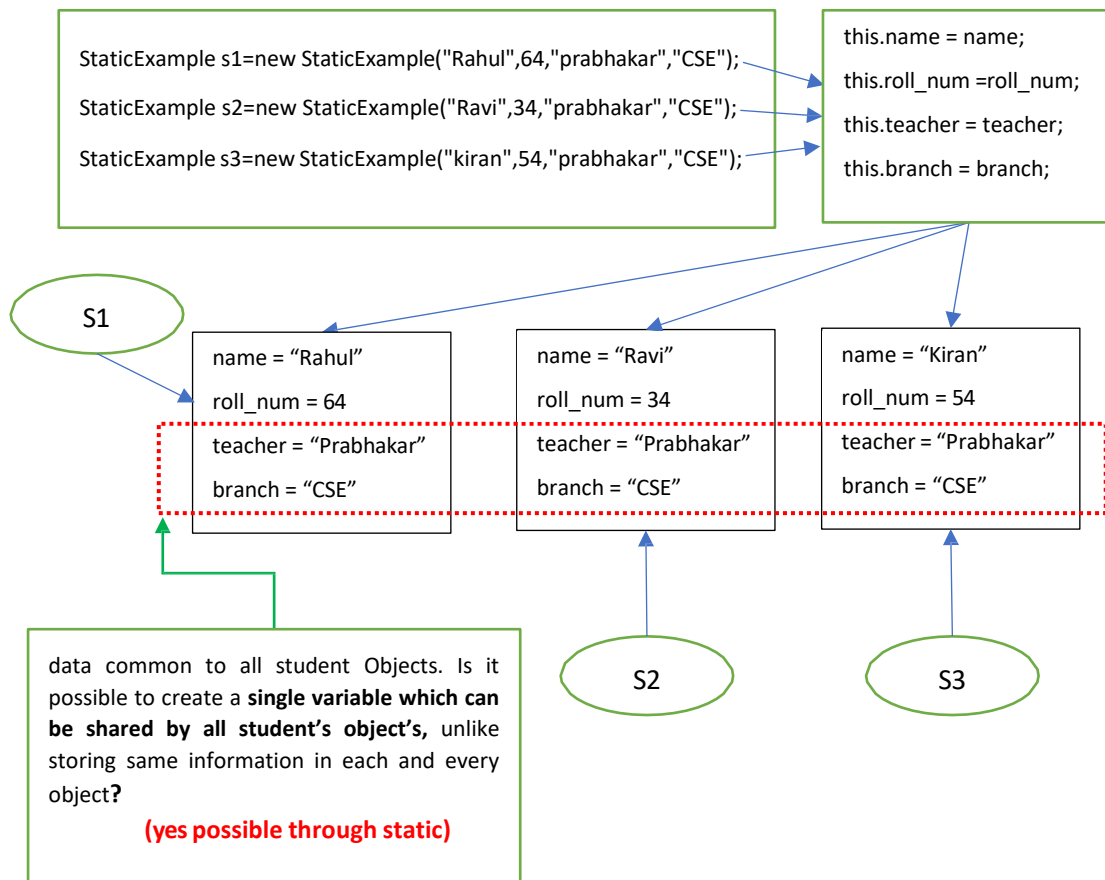
```
{
    System.out.println("name is "+name+" roll_num is "+roll_num+" teacher is "
        +teacher+" branch is "+branch);
}

public static void main(String args[])
{
    StaticExample s1=new StaticExample("Rahul",64,"prabhakar","CSE");
    StaticExample s2=new StaticExample("Ravi",34,"prabhakar","CSE");
    StaticExample s3=new StaticExample("kiran",54,"prabhakar","CSE");

    s1.displayDetails();
    s2.displayDetails();
    s3.displayDetails();
}
}
```

OutPut:

name is Rahul roll_num is 64 teacher is prabhakar branch is CSE name is
Ravi roll_num is 34 teacher is prabhakar branch is CSE name is kiran
roll_num is 54 teacher is prabhakar branch is CSE



Like this, if I'm going to create total of 195 student objects. then lot memory will be misused to

Object Oriented Programming Through Java (Unit-I)

store redundant(repeated) information. As a good programmer we should always care about memory and execution time. Either it may be 'program statements' (or) 'data', we should never repeat.

static variables:

When a variable is declared as static, then a single copy of variable is created and shared among all objects.

For the above program, to store student details declare the properties teacher and branch as static and implement the same functionality

```
public class StaticExample
{
    String name; int
    roll_num;
    static String teacher = "Mr. Prabhakar";
    static String branch = "CSE";

    StaticExample(String name, int roll_num)
    {
        this.name = name; this.roll_num =
        roll_num;
    }

    void displayDetails()
    {
        System.out.println("name is "+name+" roll_num is "+roll_num+" teacher is
        "+StaticExample.teacher+" branch is "+StaticExample.branch);
    }

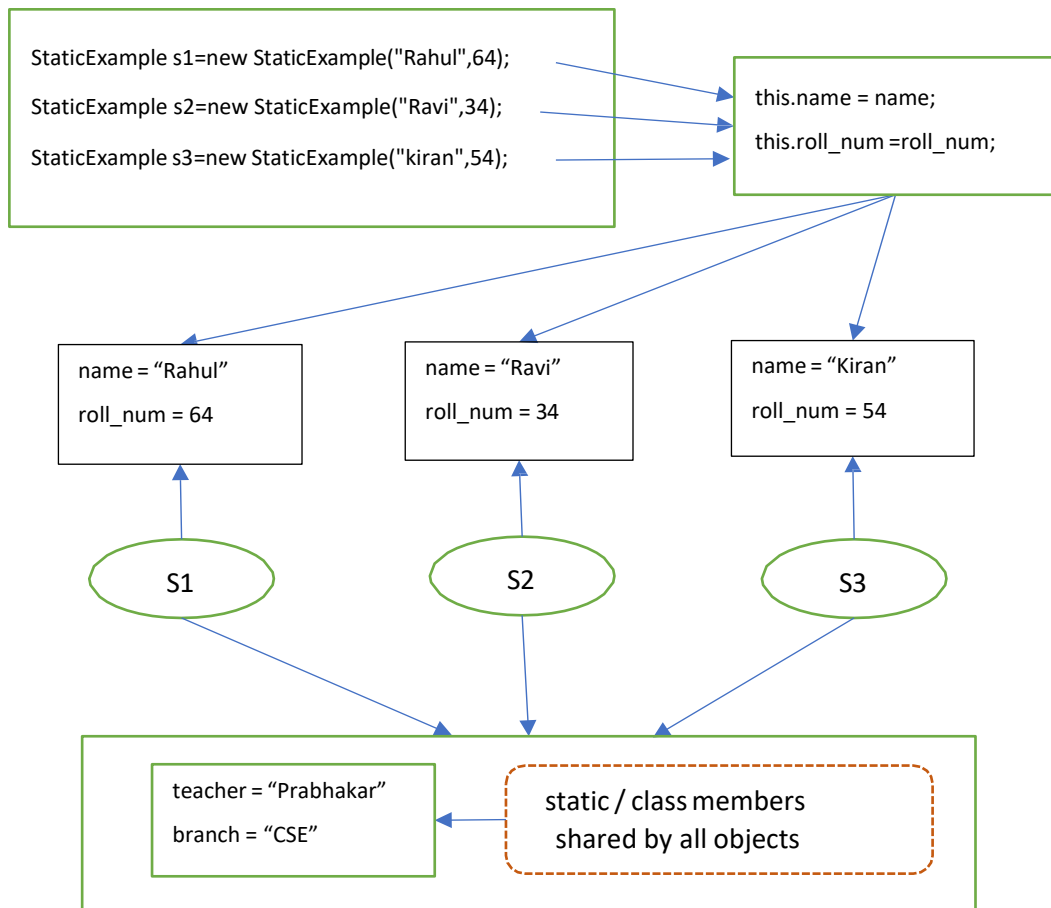
    public static void main(String args[])
    {
        StaticExample s1=new StaticExample("Rahul",64);
        StaticExample s2=new StaticExample("Ravi",34);
        StaticExample s3=new StaticExample("kiran",54);

        s1.displayDetails();
        s2.displayDetails();
        s3.displayDetails();
    }
}
```

OutPut:

name is Rahul roll_num is 64 teacher is Mr. Prabhakar branch is CSE name is Ravi
roll_num is 34 teacher is Mr. Prabhakar branch is CSE name is kiran roll_num is 54
teacher is Mr. Prabhakar branch is CSE

Object Oriented Programming Through Java (Unit-I)



Object Oriented Programming Through Java (Unit-I)

In the above program, member data teacher and branch are declared static, so only one variable for each teacher and branch will be created at class level. This static data will be shared by all student objects. It is programmer responsibility to identify which data (or) methods will be common to all objects and mark them as static, whereas data specific for individual objects should be non-static. Here rollnum and name of student cannot be shared they are individual for each student object called as instance (or) object data.

Note:

1. static variables can be accessed from static blocks and static methods only. static members can be accessed
 - i) directly without an object
 - ii) with class name
 - iii) using object
2. static members cannot invoke or access non static members.

static methods:

A static method can be invoked without creating an instance of class, they can be accessed directly or by class name.

A static method can access static data members only.

write a program with static method display, it accepts an integer number and print the square value of it, call the method display from main function by passing value 5.

```
class StaticExample
{
    static void display(int n)
    {
        int res;
        res = n*n;
        System.out.println("square value of "+ n +" is "+res);
    }

    public static void main(String args[])
    {
        display(5);
    }
}
```

OutPut:

square value of 5 is 25

static block:

whenever a java class file is executed, static block and static methods will be loaded.

From static blocks and static methods, static block will be executed first. Later among the static methods JVM will start executing main method with following signature

```
public static void main(String args[ ])
```

static blocks are used to initialize the static variables.

Write a program with any print statement in static block.

```
class StaticExample
{
    static
    {
        System.out.println("static block will be executed first");
    }
    public static void main(String args[])
    {
        System.out.println("main will be executed after static block");
    }
}
```

OutPut:

static block will be executed first.

main will be executed after static block.

this keyword:

‘this’ is a reference, that refers to the current class object.

write a program to initialize instance variables in constructor.

```
class Example
{
    int a;
    float b;

    Example(int j, float k)
    {
        a = j;
        b = k;
    }

    public static void main(String args[ ])
    {
        Example obj = new Example(5,5.4f);
        System.out.println(obj.a); System.out.println(obj.b);
    }
}
```

OutPut:

5
5.4

what if variables declared inside constructor j, k are named with same name as instance variables i.e., a, b. then how to initialize instance variables ?

A Final can be applied for variables, methods, and class.

Java final variable

If you make any variable as final, you cannot change the value of final variable(It will be constant).

Write a program using final variable, after initializing it with a value. try to update the variable again. mention the compile time error that occurs while modifying final variable.

```
class Parent
{
    final int i = 65;
    void display()
    {
        i = 60;
        System.out.println(i);
        System.out.println("method display of class Parent");
    }
    public static void main(String args[])
    {
        Parent ob = new Parent(); ob.display();
    }
}
```

Error :

java:6: error: cannot assign a value to final variable i i = 60;

Write a program, with method declared final and try to override that in child class. mention compile time error while you do that.

Note : we cannot override a final method in child class

```
class Parent
{
    final void display()
    {
        System.out.println("This method cannot be overridden in child class");
    }
}

public class Child extends Parent
{
    int i = 80;
    void display()
    {
        System.out.println("this will not executed, as we cannot override final method");
    }
    public static void main(String args[])
    {
        Child ch = new Child();
        ch.display(); // calls child display method
    }
}
```

Object Oriented Programming Through Java (Unit-I)

```
}
```

Error:

```
Child.java:13: error: display() in Child cannot override display() in Parent void display()
    ^
    overridden method is final 1 error
```

Write a program to extend a class that is final, mention the error while you obtain while implementing that

Note: We cannot extend a final class.

final class Parent

```
{
    final void display()
    {
        System.out.println("This method cannot be overridden in
                           child class");
    }
}
```

public class Child **extends** Parent

```
{
    public static void main(String args[])
    {
        Child ch = new Child();
    }
}
```

```
Child.java:9: error: cannot inherit from final Parent public class Child extends
Parent
```

```
    ^
1 error
```

Abstract classes

Abstract method: A method which is declared as abstract and does not have implementation is known as an abstract method.

Syntax

```
final dataType variableName = value;           final int a =10;
static final dataType variableName = value;     static final int b = 40;
```

Operators In Java :

Operator in java is a symbol that is used to perform operation over the operands. For example: +, -, *, / etc.

There are many types of operators in java which are given below:

- o Unary Operator,
- o Arithmetic Operator,
- o shift Operator,
- o Relational Operator,
- o Bitwise Operator,
- o Logical Operator,
- o Ternary Operator and
- o Assignment Operator

Operator Precedence	
Operators	Precedence
postfix	<i>expr++ expr--</i>
unary	<i>++expr --expr +expr -expr ~ !</i>
multiplicative	<i>* / %</i>
additive	<i>+ -</i>
shift	<i><< >> >>></i>
relational	<i>< > <= >= instanceof</i>
equality	<i>== !=</i>
bitwise AND	<i>&</i>
bitwise exclusive OR	<i>^</i>
bitwise inclusive OR	<i> </i>
logical AND	<i>&&</i>
logical OR	<i> </i>
ternary	<i>? :</i>
assignment	<i>= += -= *= /= %= &= ^= = <<= >>= >>>=</i>

1. Assignment Operator (=):

- The assignment operator is used to assign a value to a variable. For example, `int a = 5;`

2. Basic Arithmetic Operators:

- **Addition (+):** Adds two numbers. Example: `a + b`.
- **Subtraction (-):** Subtracts one number from another. Example: `a - b`.
- **Multiplication (*):** Multiplies two numbers. Example: `a * b`.
- **Division (/):** Divides one number by another. Example: `a / b`.
- **Modulus (%):** Returns the remainder of a division. Example: `a % b`.

3. Increment (++) and Decrement (--) Operators:

- **Increment (++):** Increases a variable's value by 1. It can be used as a prefix (`++a`) or postfix (`a++`).
- **Decrement (--):** Decreases a variable's value by 1. It can also be used as a prefix (`--a`) or postfix (`a--`).

4. Ternary Operator (? :):

- A shorthand for the if-else statement. It takes three operands: a condition, a result for true, and a result for false. Example: `int result = (a > b) ? a : b;` assigns `a` to `result` if `a` is greater than `b`, otherwise assigns `b`.

5. Relational Operators:

- **Greater Than (>):** Checks if the left operand is greater than the right. Example: `a > b`.
- **Less Than (<):** Checks if the left operand is less than the right. Example: `a < b`.
- **Greater Than or Equal To (>=):** Checks if the left operand is greater than or equal to the right. Example: `a >= b`.
- **Less Than or Equal To (<=):** Checks if the left operand is less than or equal to the right. Example: `a <= b`.
- **Equal To (==):** Checks if two operands are equal. Example: `a == b`.
- **Not Equal To (!=):** Checks if two operands are not equal. Example: `a != b`.

6. Boolean Logical Operators:

- **AND (&&):** Returns true if both conditions are true. Example: `(a > b) && (c > d)`.
- **OR (||):** Returns true if at least one condition is true. Example: `(a > b) || (c > d)`.
- **NOT (!):** Inverts the value of a boolean. Example: `!(a > b)`.

7. Bitwise Logical Operators:

- **AND (&):** Performs a bitwise AND operation. Example: `a & b` (operates on each bit of the operands).
- **OR (|):** Performs a bitwise OR operation. Example: `a | b`.
- **XOR (^):** Performs a bitwise XOR operation. Example: `a ^ b`.
- **Complement (~):** Inverts all the bits of the operand. Example: `~a`.

Introduction to Control Statements :

Control statements direct the flow of a program based on conditions or repetitions. These include decision-making statements like if, switch, and loops like for, while, and do-while.

1. if Expression

The if statement is used to execute a block of code only if a specified condition is true.

Example:

```
int age = 18;
if (age >= 18) {
    System.out.println("You are eligible to vote.");
}
```

2. Nested if Expressions

A nested if is an if statement inside another if statement. It is used to test multiple conditions.

Example:

```
int age = 20;
if (age >= 18) {
    if (age >= 21) {
        System.out.println("You are eligible for PSU Job.");
    }
    System.out.println("You are eligible to vote.");
}
```

3. if-else Expressions

The if-else statement is used to execute one block of code if a condition is true and another block if the condition is false.

Example:

```
int num = 10;
if (num % 2 == 0) {
    System.out.println("The number is even.");
} else {
    System.out.println("The number is odd.");
}
```

4. Ternary Operator (? :)

The ternary operator is a shorthand for if–else statements. It takes three operands and returns a value based on a condition.

Example:

```
int a = 5, b = 10;

int max = (a > b) ? a : b;

System.out.println("Maximum value: " + max);
```

5. switch Statement

The switch statement is used to select one of many code blocks to execute. It's often used for evaluating an expression against multiple values.

Example:

```
int day = 3;

switch (day) {

    case 1:

        System.out.println("Monday"); break;

    case 2:

        System.out.println("Tuesday"); break;

    case 3:

        System.out.println("Wednesday");

        break;

    default:

        System.out.println("Invalid day");

}
```

Iteration Statements

Iteration statements are used to execute a block of code repeatedly. The main types are while, do-while, for, and for-each.

1. while Expression

The while loop executes a block of code as long as a specified condition is true.

Example:

```
int i = 0;

while (i < 5) {

    System.out.println(i); i++;

}
```

```
}
```

2. do-while Loop

The do-while loop is similar to the while loop, but it executes the block of code once before checking the condition.

Example:

```
int i = 0; do {  
    System.out.println(i); i++;  
} while (i < 5);
```

3. for Loop

The for loop is used to execute a block of code a specific number of times. It is typically used when the number of iterations is known.

Example:

```
for (int i = 0; i < 5; i++) {  
    System.out.println(i);  
}
```

4. Nested for Loop

A nested for loop is a for loop inside another for loop. It is often used for working with multi-dimensional data structures like matrices.

Example:

```
int k = 1;  
for (int i = 1; i <= 3; i++) {  
    for (int j = 1; j <= 3; j++) {  
        System.out.println(k); k++;  
    }  
}
```

5. for-each Loop

The for-each loop (also known as the enhanced for loop) is used to iterate over elements in an array or collection.

Example:

```
int[] numbers = {1, 2, 3, 4, 5};  
  
for (int num : numbers) {  
    System.out.println(num);  
}
```

6. break Statement

The break statement is used to exit a loop or switch statement.

Object Oriented Programming Through Java (Unit-I)

Example:

```
for (int i = 0; i < 10; i++)
```

```
{
```

```
    if (i == 5)
```

```
    {
```

```
        break;
```

```
    }
```

```
    System.out.println(i);
```

```
}
```

7. continue Statement

The continue statement is used to skip the current iteration of a loop and move to the next iteration.

Example:

```
for (int i = 0; i < 5; i++)
```

```
{
```

```
    if (i == 3) {
```

```
        continue;
```

```
    }
```

```
    System.out.println(i);
```

```
}
```

-----*****-----